

指尖上的智能

端侧行为感知与意图识别的工程实践

周刘纪

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon

上海站

全球软件开发大会

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon

北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

用户在京东的主要行为路径



目录

01 现状和价值

02 关键挑战

03 工程实现

04 业务实践

01 现状与价值

电商场景中用户行为特点

01

决策链路短

目的性强 | 路径连续 | 回退少

- 搜索栏输入精确词 → 快速进入商详
- 主图/价格/配送 → 立即购买或加购
- 购物车勾选 → 去结算 → 收银台

02

行为碎片化与高频化

事件量大 | 频次高 | 分布零散

- 首页 / 逛 / 购物车 / 消息 / 我的频繁切换
- 秒送 / 特价 / 新品频道左右滑动切换
- Feeds 连续下滑, 多商品点击、重复加购/收藏

03

行为与环境强相关

时间 | 位置 | 设备 | 场景共同影响

- 工作日白天 vs 晚间/周末: 频道偏好变化
- 不同城市: 推荐与促销入口点击存在差异
- WiFi / 移动网络、大促场景: 行为密度变化

04

无明确诉求, 随机浏览

无固定目标 | 路径跳跃 | 转化弱

- 不搜索, 首页 Feeds / 「逛」Tab 持续浏览
- 九宫格随机点击分类入口; 商详浏览后退出
- 收藏 / 点赞 / 分享较多, 但缺少加购等转化

端侧数据价值

原始事件流
↓
行为模式

① 实时捕获

点击、滑动、停留、Tab切换
页面路径、加购/收藏/购买等信号

② 时序 & 模式分析

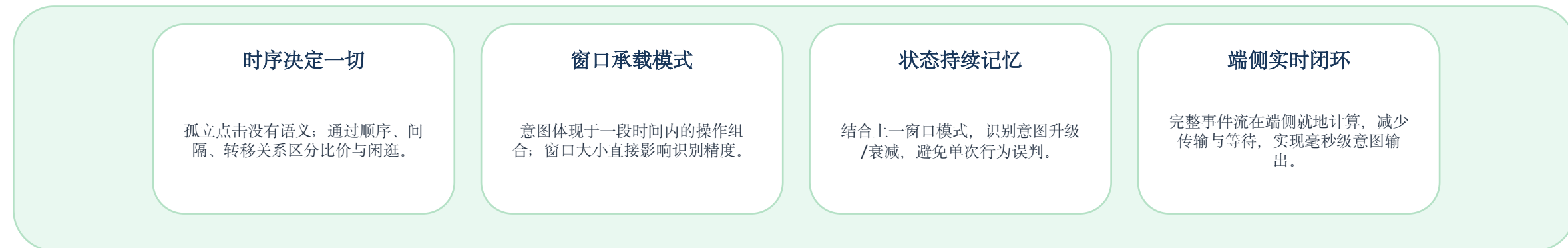
点击间隔、滑动速度、操作顺序
路径重复度、状态转移概率

③ 模式 → 意图

短决策链 / 高频碎片 / 环境依赖
随机浏览 → 为意图判断提供原始燃料

行为产生数据 · 数据提炼模式 · 模式推断意图

端侧行为不是孤立、而是连续事件流



端侧实时意图分析的优势

01 实时感知

连续捕获 · 不丢细节

Click

Swipe

Stay

Back

→ 时间戳排序 → 连续事件流

02 实时计算

就地聚合 · 边产生边计算

30s / 60s / 5min

Window

特征向量

Feature

行为模式

Pattern

→ 意图类型 / 强度 / 置信度

03 实时决策

毫秒响应 · 抢占转化窗口

弹窗

引导

推荐

优惠

意图信号 → UI Action
毫秒级触达，无需云端回传

→ 抢占用户最强意图窗口

为什么必须在端侧完成实时分析？

信息层级

云端可见

端侧独有

业务价值

离散结果

加购 / 点击 / 下单

✓

只能看到结果

连续轨迹

部分埋点 / 延迟上报

✓ 完整保留

识别真实意图

时序关系

难恢复

✓ 间隔 / 顺序 / 回退

判断意图强度

核心：端侧直接接触“行为发生现场”，既获得最完整时序，也避免海量原始事件先上云再计算。

端侧实时分析的技术闭环

行为产生

数据捕获

窗口计算

模式识别

意图输出

实时行动

最实时 · 最完整 · 数据量可控 · 可直接决策

端侧价值：数据不必“先上报、再分析、再决策”，而是在行为发生的同时完成 感知 → 计算 → 决策。

03 关键挑战

算力资源、数据洪峰、工程挑战

算力资源：端侧不是独占的

核心矛盾 意图分析与页面渲染 / 图片解码 / 列表滑动 / 网络请求 / 业务逻辑争抢 CPU 时间片、内存带宽与功耗预算

① 主要算力竞争者

页面渲染

UI 线程 + 渲染管线持续占用

图片解码

商品图 / 封面缩放消耗 CPU、带宽

列表滑动

布局计算 + View 复用依赖主线程

网络 / 业务

序列化、加密、动画与状态管理

② 资源竞争后果

CPU

时间片被抢占

Memory

带宽争抢 / 整体变慢

Power

发热与电量消耗叠加

Latency

调度波动 → 意图延迟不可控

③ 端侧资源治理

算力分级

P0 结算识别优先; P2 长时统计可让渡

自适应调度

滑动时暂停 / 降级; 停留时利用空闲算力补算

熔断降级

CPU > 80% / 温度过高 → 跳过非关键任务

动态调度闭环

设备状态

任务优先级

资源监测

动态执行

降级 / 恢复

数据洪峰：高频事件如何不压垮端侧

核心矛盾

全量行为事件 + 业务埋点 + 设备 / 传感器数据高频进入，必须在毫秒级完成过滤、聚合与特征提取，同时控制内存与 CPU

① 数据洪峰从哪里来？

行为事件

点击 / 滑动 / 停留 / 切换 / 回退
高频连续

业务埋点

曝光 / 加购 / 下单 / 支付
类型多、格式异

设备数据

电量 / 网络 / 存储 / 亮度
动态变化

传感器

陀螺仪 / 加速度 / 环境光
连续采样

② 洪峰带来的计算压力

全量进入

事件持续涌入 → 队列 / 内存持续增长

毫秒处理

采样 → 聚合 → 特征 → 模式匹配

异构归一

坐标 / JSON / 三轴数据需要统一时序，统一接入

低端机风险

CPU / 内存持续占用 → 卡顿 / ANR / OOM

③ 三道“减压阀”

分级过滤

核心事件高保真；辅助事件降采样，减少无效入队

窗口聚合

滑动窗口增量统计，把海量事件压缩为特征向量

内存池化

对象缓存池化，尽可能复用内存数据，避免内存频繁创建销毁带来稳定性波动

洪峰处理链

原子事件

分级过滤

滑动窗口

增量聚合

实时特征

数据洪峰 → 分级过滤 + 窗口聚合 + 内存池化 → 不丢关键事件、不爆内存、不拖主线程

工程挑战：工程架构复杂度

核心矛盾

iOS / Android / Harmony × 规则 / 模型 / 统计 × 首页 / 商详 / 购物车 / 逛等场景同时触发，资源竞争与性能波动难以预测

① 架构复杂性：三层乘法

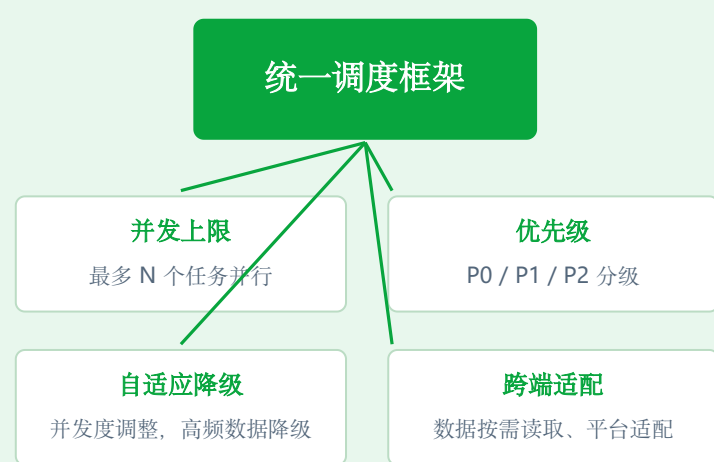


跨端序列化、调度、生命周期与策略切换进一步放大复杂度

② 没有统一治理会发生什么？



③ 统一资源编排层



统一治理闭环



工程架构 → 统一调度 + 并发控制 + 自适应降级 + 跨端适配 → 让复杂端侧智能稳定、可控、可预期

04 工程实现

编译器 · 运行时引擎 · 业务框架的深度技术设计

产品架构

端云协同
·
实时智能
·
体验增长

智能服务

场景化服务封装

智能触点系统

行为捕获 → 模式匹配 → 执行触点 → 触达执行

智能预测服务

行为捕获 → 历史数据分析 → 预测下一步行为

意图识别服务

行为流输入 → 意图推理 → 标签输出

业务框架

工程化基础能力

事件中心

统一事件接入、清洗、时序对齐、全链路埋点

策略工程

策略编排、IDE 开发、A/B 实验、灰度发布、回滚

特征工程

特征提取、交叉、归一化、复用与实时更新

核心引擎

高性能计算引擎

DPython

高性能 Python 执行引擎

DongCEP

复杂事件处理与实时规则匹配

DongFlow

流式计算与窗口聚合

DongDT

轻量级决策与模型推理

DongNNE

神经网络推理与模型加速

端侧设备

全端覆盖触点

原生 App

iPhone / Android / 鸿蒙

跨端与容器

小程序 / TN / Taro

统一接入

一套能力，多端运行

DPython: 编译期做重, 运行时做轻

设计目标

动态策略下发

减少客户端发版依赖

Python-like 表达

条件 / 循环 / 集合 / 后处理

多引擎统一决策

CEP / Flow / DT / NNE / 策略

端侧可控

高性能 / 高稳定性 / 可治理 / 轻量

核心判断

不直接采用 CPython

完整动态类型、对象协议、模块与标准库 → 端侧成本高, 并发度低, 包体积大, 性能差

DPython 取舍

保留 Python-like 语法子集, 不追求完整 Python 兼容

重新设计

静态强类型, 高性能基础库, 自定义字节码
强类型编译器 + 轻量寄存器虚拟机 + 多策略内存缓存

运行时

DPython Runtime

Bytecode Verify → Register VM Run

Feature / Event / KV

DongCEP / DongFlow

DongDT / DongNNE

Business Action

Bridge 白名单: 限定数据、函数、模型能力与业务 Action 的可访问边界

一句话: 业务看到的是 Python 的表达能力, 端侧执行的是静态约束的强类型、寄存器式、可治理的高性能字节码。

DPython 编译器：强类型驱动高性能字节码生成

核心理念

源码复杂度 → 编译期消化 | 字节码紧凑化 → 运行时直接执行 | 类型与寄存器信息 → 提前固化

1 源码

Source

Python 子集源码
类型注解 / Builtin 签名

2 强类型编译前端

Typed Frontend

AST / 脱糖 / 类型系统
Type Check / 编译诊断

3 寄存器化 IR

Register IR

作用域 / 基本块
寄存器分配 / 类型化 IR

4 专用指令集

Custom ISA

Typed Opcode
控制流 / Call 指令

5 汇编与符号

Assembler

符号管理 / 地址修正
指令优化 / 寄存器统计

6 字节码封装

Packaging

Header / Code / Const
Dynamic Symbol / Debug

强类型 · 静态检测 · 符号系统

- 变量 / 参数 / 返回值 / 容器元素类型固化，校验 List / Dict / Tuple 元素、类实例与字段类型
- 类型确定后选择专用指令，减少运行时动态分派
- 基础值直接进寄存器，仅在对象边界按需装箱

寄存器复用 · 高性能字节码

- 寄存器式 IR，减少 Push / Pop 与临时值搬运
- 紧凑寄存器 / 常量 / 跳转偏移编码
- 数据依赖更清晰，便于类型专用指令生成
- 降低解释器中间状态管理成本

自定义 32-bit 指令集

- 固定 32-bit: Opcode + Target + Source1 + Source2 / Immediate
- Int / Float / Object 专用算术、比较与跳转
- 覆盖 List / Dict / Tuple / Iterator / Slice / 类实例

编译优化 · 运行时轻量

- 直接加载 Code / Const / Symbol 并执行
- Builtin 函数统一进入 Dynamic Symbol
- 运行时仅做符号绑定，不做复杂签名匹配

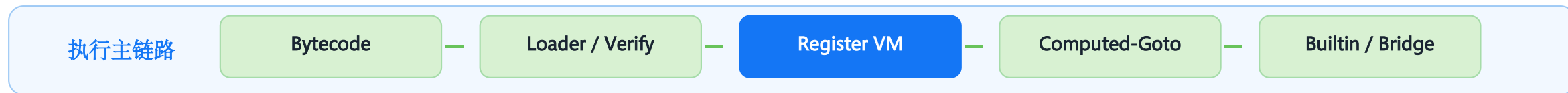
编译器做重

类型建模 · 类型检测 · 寄存器分配 · 指令选择 · 地址解析 · 优化 · 封装

运行时做轻

加载 · 动态符号绑定 · Register VM 执行

DPython Runtime: 高性能、高并发、跨平台



① 执行内核：把解释成本压到最低

- 寄存器式 VM：基础值直接驻留 64-bit Register
- 类型专用 Opcode：int / float / object 分离，编译期确定操作数类型，运行时避免动态分派。
- opcode → Jump Table → Handler，缩短公共 dispatch 路径并提升指令局部性。

② 内存体系：复用优先、对象按需创建

- Thread-Local Executor：复用 Runtime / Frame / Cache，避免重复初始化与锁竞争
- Frame Pool：函数调用帧循环复用；
- Object Pool：复用 Int / Float / String / List / Dict / Tuple 等高频对象。
- Constant Cache：常量字符串与只读对象复用；减少重复分配、拷贝和 GC/释放压力。

③ 并发模型：线程隔离 + 分片共享

- Executor 与线程绑定：内存隔离，无状态，无锁。
- 并发设计核心：本地状态无锁化，共享状态分片化，生命周期原子化。

④ 安全与稳定：字节码可验证、能力可治理

- 能力可控：脚本只能访问注册的数据、Builtin、模型能力与 Business Action。
- 资源边界：寄存器、调用栈、对象池、缓存容量均有上限，避免脚本放大资源消耗。
- Runtime Error / Crash Dump / 执行状态可回传，支持错误定位与灰度治理。
- 故障隔离原则：脚本执行失败不阻塞宿主业务主流程，运行时异常可快速退出。

DPython vs CPython | 端侧运行时多维度对比

5.2×

单线程整体

15.5×

典型场景

240.93×

并发场景

维度	CPython	DPython
设计目标	完整 Python 语义 通用生态	端侧策略高性能 轻量·高稳定性·可治理
性能表现	通用动态执行路径	单线程约 5.2× 典型场景约 15.5×
并发表现	传统 GIL 模型 限制字节码并行	Thread-Local Executor 并发实测提升 240.93×
类型与指令	动态类型 通用字节码	强类型·寄存器 VM 类型专用指令
内存与缓存	通用对象模型 运行时缓存	基础值直存 Executor / Frame / Object / 常量复用
能力边界	完整语法、标准库 第三方生态	Python-like 子集 Bridge 白名单治理
包体与跨端	完整解释器与标准库 > 1M	轻量 C/C++ 核心 统一支持多端 < 200k

CPython 优先保证语言完整性与生态;

DPython 通过“强类型编译 + 寄存器 VM + 专用指令集 + 多级缓存”，换取端侧更高的性能、并发能力、轻量性与可治理性。

DPython 的核心优势

更快

强类型前移 + 寄存器 VM + 类型专用指令;
基础值直存 + Computed-Goto, 缩短执行路径

更高并发

Thread-Local Executor 隔离执行上下文;
Domain + 16 Shard 降低共享缓存竞争。

更轻量

不携带完整标准库与模块系统;
编译器做重, 端侧只保留加载、执行与 Bridge。

更稳定 /
可治理

编译期错误拦截 + 字节码校验 + 资源边界;
Bridge 白名单控制数据与系统能力访问。

设计逻辑

编译期确定性 → 运行时低成本
受控能力边界 → 端侧安全稳定

05 业务实践

■ 首页内容频道刷新——意图分层驱动的实时刷新决策

场景痛点

用户从商详 / 搜索结果返回首页时，直播入口内容可能已过时；核心矛盾是“刷新要不要做、刷新到什么程度”。

全量刷新 → 浪费流量、打断浏览；不刷新 → 内容陈旧、错失转化窗口；传统固定策略无法根据真实意图差异化决策。

端侧意图分层：行为 → 意图 → 刷新动作

毫秒级本地判断 | 无网络往返

01 浅层浏览

快速划过 · 停留短 · 无深度交互

不刷新

避免打扰

02 深度探索

停留较长 · 看图/评论 · 未进入交易

轻量刷新

保持内容新鲜

03 决策评估

看优惠 · 比价格 · 进店了解

精准刷新

匹配目标品类

04 交易行动

加购 · 下单 · 意图高度明确

即时刷新

承接转化窗口

关键：意图判定在端侧实时完成，行为发生 → 刷新决策仅数毫秒。

端侧实时决策架构

特征生产与规则判定解耦，避免重复订阅

用户微行为

停留 / 加购 / 优惠 / 图片

特征中心

共享写入 · 统一读取

规则包

只读特征 · 意图判定

刷新决策

不刷 / 轻刷 / 精准 / 即时

首页场景：纯内存模式 · 零磁盘开销

效果

精细化刷新：高意图用户获得实时体验，低意图场景避免无谓刷新，实现“内容新鲜度 × 流量效率 × 用户体验”平衡，从“全量刷新 / 不刷新”到“按用户意图分层刷新” | 让刷新资源与转化窗口精准匹配

风控：端侧微行为感知与实时风险特征工程

场景痛点

传统云端风控存在三类结构性盲区：延迟高、感知浅、隐私敏感。

01 延迟高

上报 → 排队 → 计算 → 回传

02 感知浅

曝光 / 点击等粗粒度信号

03 隐私敏感

原始行为明细上云合规成本高

端侧如何感知：实时风险特征工程管线

微行为 → 特征 → 风险标签

01

意图识别

搜索关键词偏好 · 品类倾向

实时分类 / 打标

高风险意图模式

02

路径还原

页面跳转顺序 · 浏览路径

实时还原 / 分析

异常行为链路

03

设备状态

端侧传感器 · 操作状态

实时检测 / 判断

机器行为直接证据

04

行为深度

交互丰富度 · 浏览深度

聚合微行为计数

结构化风险特征

关键：全部加工在端侧实时完成，微行为发生 → 特征产出仅数毫秒。

端侧独有价值

云端无法直接获得的信号

传感器信号

检测非人工操作状态

页面级微行为

还原细粒度交互轨迹

行为序列

识别异常链路 & 路径

隐私边界

原始明细不出端

仅输出脱敏风险标签 + 聚合特征

端云协同链路

微行为发生

端侧特征工程

风险标签 / 聚合特征

云端模型 / 决策

毫秒级

补充云端无法获取的传感器信号与微行为序列，使交易反欺诈、营销防刷具备毫秒级响应能力。

原始行为明细不出端，仅上传脱敏风险标签与聚合特征 | 实时性、信号深度与合规边界同步提升，从“云端粗粒度信号”到“端侧微行为特征” | 毫秒级识别风险，构建更深、更快、更合规的风控能力

搜推——端侧意图信号增强

场景痛点

服务端知道“搜了什么、点了什么”，却看不到“搜完之后做了什么”，导致最关键的即时意图信号无法及时反馈。

快速换词重搜？深入浏览多款商品？反复对比？跨品类漂移？这些微行为因链路延迟，往往要到下一次请求才生效。

端侧如何感知：三类即时意图信号

近 7 天行为历史 | 实时查询

01

搜索后浏览深度

快速返回 ↔ 深入浏览多个商详

深度越深，品类意图越明确

02

对比与回搜行为

同类反复对比 ↔ 搜索后换词重搜

对比=决策评估 | 回搜=需求未满足

03

跨品类漂移

A 品类 → B 品类

实时修正推荐方向

核心：微行为不是“历史记录”，而是随当前会话实时变化的动态意图信号。

端侧信号 → 搜推消费

一次加工，多处消费 | 跨场景特征复用

搜索埋点

事件中心自动订阅

端侧行为历史

本地落盘 | 近 7 天

特征中心

停留 / 购买意图等

意图标签

品类 / 对比 / 回搜

搜推召回排序

即时补充特征

实时链路

行为发生

端侧查询

意图识别

召回/排序

秒级 → 毫秒级

成效

搜索后的微行为不再“下次请求才生效”，而是即时反馈当前推荐决策，提升搜索推荐的即时相关性。

端侧意图信号实时补充云端召回排序特征 | 行为发生 → 特征可用：秒级压缩至毫秒级，从“只知道用户搜了什么”到“实时理解用户搜完之后做了什么” | 毫秒级补充即时意图

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会